



RAFFLES INSTITUTION

H2 Computing (9569)

2025 Year 5

Tutorial S2A: Programming Elements (Solutions)

Discussion Questions

1.

(Marking: 1 for condition and 1 for both parts of test data x2)

Condition being tested	Array data	Search integer
Search integer occurs once	5 8 5 7 2 3 6 4	7
Search integer occurs more than once	5 8 5 7 2 3 7 4	7
Array data contains all same integer (= or $\neq$ Search)	2 2 2 2 2 2 2 2	2
Array empty		2
Search integer does not exist	5 8 5 9 2 3 9 4	

Other possibilities can be credited

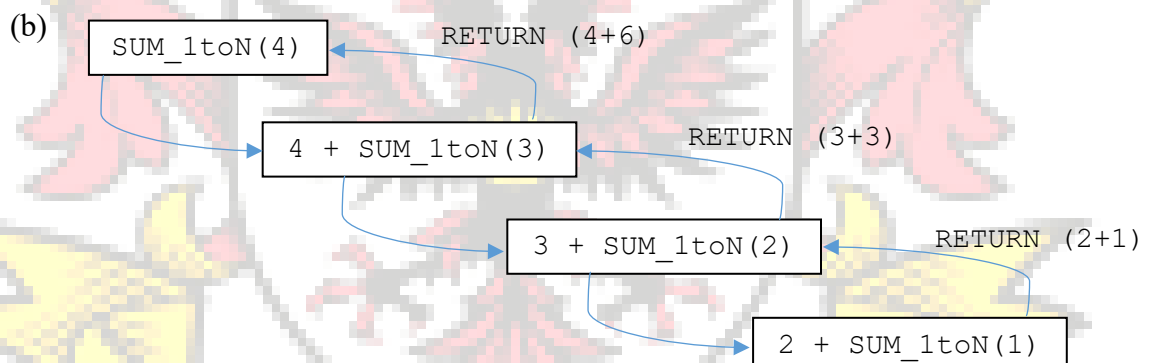
2(a)

I	X	Y	Z
	2	4	1
1			2
2			4
3			8
4			16

(b) Calculate X to the power of Y.

3. (a) Pseudocode:

```
FUNCTION SUM_1toN(N: INTEGER) RETURNS INTEGER
    IF N = 1 THEN
        RETURN 1
    ELSE
        RETURN N + SUM_1toN(N-1)
    ENDIF
ENDFUNCTION
```



(c) Total number of times SUM_1toN(4) is called is 1.

(d) Total number of times SUM_1toN(4) calls itself is 3.



RAFFLES INSTITUTION
H2 Computing (9569)
2025 Year 5

4

Line number	Explanation of error	Correction of error
Line 3	Syntax error. Missing quotes for string.	<code>print("Welcome to DAISO")</code>
Line 5	Runtime error. Str type multiply with float type.	<code>cost = float(quantity)*unitprice</code>
Line 6	Logic error. gst should be 0.07 since it is 7%	<code>gstcost = cost*gst/100</code>

- 5(a) Length check
 (b) Range check
 (c) Type check
 (d) Format check

- 6(a) Ticket_prices should be between \$650 to \$1200
 (b) Flight_number should start with EK, followed by three digits.
 (c) Airport_code should be 3 characters.

7

Input (test cases)	Purpose (test for ...)	Expected output
9, 11, 33	Normal data	Accepted
8, 10, -1	Abnormal data	Rejected
1, 79 (assume max width of screen is 80, largest odd number is 79)	Extreme data	Accepted

8 SV = 75

Array

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
13	27	33	49	55	67	75	79

S	E	M	Array[M]	F	P
0	7	3	49	False	-1
4	7	5	67		
6	7	6	75	True	6



RAFFLES INSTITUTION

H2 Computing (9569)

2025 Year 5

9(a) Function that calls itself in its definition. With a stopping condition to stop calling itself. Each call to itself must tend towards stopping condition. A call stack is required – to store return address, parameters and local variables.

(b)(i)

	numList				first	new
LENGTH (numList)	0	1	2	3	first	new
4	3	5	10	2	3	
3	5	10	2		5	
2	10	2			10	
1	2				2	
0	2					
	2	10				
	2	10	5			
	2	10	5	3		[2, 10, 5, 3]

(b)(ii) reverse a list

10(a) A variable in parentheses of the function used to hold a value that is sent to the function, acts like an input value to the function.

For example, the Word is a parameter, that will hold a value sent to this function IsInOrder.

10(b) Relational operator to check if left side is equals to right side.

Assignment operator for assigning value to left side variable.

10(c) “FoRtY” is in different cases. This makes it impossible to compare the letter, because an uppercase letter and a lowercase letter are not equal even when they are the same letter. To make it possible to compare, change all letters to a single case, either upper or lower.

10(d) A function that calls itself, with a stopping condition to stop calling itself. Every call to itself tends towards the stopping condition.

IsInOrder calls itself in line 10. Stopping condition is in line 2, when the function stops calling itself. The length of parameter Word will get shorter by 1 each time the function is called, as line 6 show that the argument passed into the recursive function gets shorter with each call.

10(e) True will be returned. Lines 1, 2, 3 will be executed.

10(f) line 2 will be false, so run line 4, then line 5, which gives runtime error, since there is no first character in Word.

10(g) Call IsInOrder(“APE”), where argument is “APE”. Diagrams show the steps taken, where IsInOrder(“PE”) will be called in line 10. After returning from calling IsInOrder(“PE”), the value False will be returned.



RAFFLES INSTITUTION
H2 Computing (9569)
2025 Year 5

IsInOrder("APE")

Line 2 condition not met

- line 4: ELSE
- line 5: FirstChar = "A"
- line 6: RestOfWord = "PE"
- line 7: FALSE
- line 9: ELSE
- line 10: Call IsInOrder("PE")
- RETURN False

IsInOrder("PE")

Line 2 condition not met

- line 4: ELSE
- line 5: FirstChar = "P"
- line 6: RestOfWord = "E"
- line 7: TRUE
- line 8: RETURN False

11(a) **Syntax** error occurs when program statements cannot be understood because they do not follow the rules laid down by the programming language. In this case, the reserved word **TO** is wrongly typed. There is no keyword **TOO**, hence compiler reported syntax error. It should read **FOR Counter = 1 TO ArraySize**.

11(b) **Sum** is declared as **INTEGER** (line 2), but the value assigned is a **REAL** number (line 4). Hence compiler reported **incompatible data type**.

11(c)(i) **NOTE**: The question states that **compilation of program was successful**, but when the code was run, the **program failed to complete** and reported an error at line 7. Hence there was a run-time error. (*Read below part (ii) first*)

ANSWER: Run time error (overflow error).

Overflow error occurs when the computer attempts to handle a number that is too large for it. Every computer has a well-defined range of values that it can represent. If during execution of a program it arrives at a number outside this range, it will experience an overflow error.

JUSTIFICATION:

An **integer** data type only uses 4 bytes.

4 bytes is 32 bits.

Hence largest number is $2^{32} - 1$, which is **4,294,967,295**.

However, if you consider **both positive and negative numbers**, then the largest number that a computer can handle is **2,147,483,647** and the smallest number is **-2,147,483,648**.

The addition of **Sum** and **Temp** in line 7 might give a value that exceed what is represented by the 4-byte integer.



RAFFLES INSTITUTION
H2 Computing (9569)
2025 Year 5

11(c)(ii) NOTE: The first statement of this question states that “*A function is to be written that returns the sum of all values held in an array ...*”

Since the program function is supposed to sum all the values, NOT to multiply all the values, the code should be changed from multiply (*) to add (+) in line 7.

Hence for line 7, the code should be changed from

Sum = Sum * Temp to **Sum = Sum + Temp.**

The error in **part(c)(i)** depends on the detection of this **logic error**. The compiler compiled successfully because the syntax was correct, i.e. there was **no problem in the syntax** for line 7: **Sum = Sum * Temp**. But the purpose was to do add all the values, and not multiply all the values. Hence, **logic error**.

11(d)(i) In black box testing, the tester only knows the inputs and what the expected outcomes should be and not how the program arrives at those outputs. The function call **TotalSum(TempArray, 1000, 5.0)** uses **TempArray** which contains random numbers, which makes it impossible to know the outcome of the function call as we don't know what actual numbers will be added to the sum. Hence not an appropriate black box test.

(ii) White box testing tests internal structures or workings of an application. The function call **TotalSum(TempArray, 10, 10.5)** has **ArraySize** of 10 and **MinValue** of 10.5. This means that the FOR-loop (line 5) will run 10 times and tested against the **MinValue** of 10.5. However **TempArray** only stores a highest value of 10.0 because it is stated that “each element of **TempArray** stores a random value between 1.0 and 10.0”. This means that the comparison made in line 7 (**IF TEMP > MinValue**) will **always** be **FALSE**, and **Sum** will forever be the initial value of 0.

11(e) NOTE: For this question, the justification must accompany the answer you state.

Since every element in the array is 1.0 and the function returns the sum of all the values, we can expect the output to be equal to the **ArraySize** if the **MinValue** is less than 1.0.

For example, if the function call is **TotalSum(TempArray, 10, 0.9)**. Expected output will be 10.0, since the value of 1.0 will be added to **Sum** 10 times. The FOR-loop in line 5 will run 10 times, and each time it is run, the condition in line 7 (**IF TEMP > MinValue**) will be TRUE, hence **Temp** will be added to **Sum** for each loop. This will give an expected output of 10.0.

Hence, any function call that tests extreme values of **MinValue** and **ArraySize** will be an appropriate black box test.

Examples:

function call **TotalSum(TempArray, 50000, 0.9)** will give an output of 50000;

function call **TotalSum(TempArray, 1, 0.9)** will give an output of 1;

function call **TotalSum(TempArray, 50000, 1.1)** will give an output of 0;

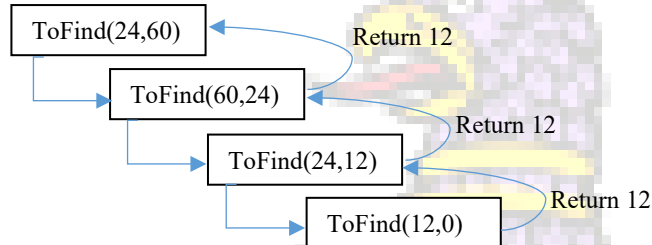


RAFFLES INSTITUTION
H2 Computing (9569)
2025 Year 5

Assignment Questions

1.

(a)



(b) Find the highest common factor of x and y.

(c) `FUNCTION ToFindIte(x: INTEGER, y:INTEGER) RETURNS INTEGER
 DECLARE Temp: INTEGER
 WHILE y <> 0
 Temp ← y
 Y ← x MOD y
 X ← Temp
 ENDWHILE
 RETURN x
ENDFUNCTION`

Alternative solution:

```
FUNCTION ToFindIte(x: INTEGER, y:INTEGER) RETURNS INTEGER
    IF x>y THEN
        small = y
    ELSE
        small = x
    ENDIF
    FOR num = 1 TO small
        IF x MOD num = 0 AND y MOD num = 0 THEN
            HCF = num
        ENDIF
    ENDFOR
    RETURN HCF
ENDFUNCTION
```



RAFFLES INSTITUTION
H2 Computing (9569)
2025 Year 5

2(a) FUNCTION GenerateCheckDigit(ISBN:STRING) RETURNS **CHAR**
 DECLARE Sum,Weight,i,CheckDigit,Remainder: INTEGER
 Weight $\leftarrow$ 10
 Sum $\leftarrow$ 0
 FOR i $\leftarrow$ 1 TO 9
 Sum $\leftarrow$ Sum + ISBN[i]*Weight
 Weight $\leftarrow$ Weight - 1
 ENDFOR
 Remainder $\leftarrow$ Sum MOD 11
 CheckDigit $\leftarrow$ 11 - Remainder
 IF CheckDigit = 10 THEN
 RETURN 'X'
 ELSE IF CheckDigit = 11 THEN
 RETURN '0'
 ELSE
 RETURN **CHAR**(CheckDigit)
 ENDIF
ENDFUNCTION

- 2(b) To validate the input data to ensure data entry or data transmission is correct.
Check digit is generated by an algorithm and added to the end of the data.
Transposition error – digits entered are switched positions, eg. 12345 becomes 12435
Transcription error – digits are entered wrongly, eg. 12345 becomes 12346